Prompt Forge Kits

PHASER 3 + VITE · AI-READY

Browser Game Starter Kit

by **Prompt Forge Kits**

Build and ship your first browser game with AI coding tools

1 Quick Start
2 Game Design Doc Template
3 Visual Theme
4 Prompt Playbook
5 Grow Your Game
6 The Fix Loop
7 Publishing Your Game
8 License

v1.0 · Phaser 3 + Vite · Works with Cursor, Claude Code, Windsurf, Lovable, Bolt
Art drawn in code · Font: Lilita One (OFL)

© 2026 Prompt Forge Kits · promptforgekits.com · @promptforgekits



Quick Start: Up and Running in 15 Minutes

Browser Game Starter Kit by Prompt Forge Kits

What you need: a computer, Node.js 18+ (LTS), Git, and an AI coding tool (Cursor, Claude Code, Windsurf, or Copilot).

Works best with AI tools that open a local folder: Cursor, Claude Code, Windsurf, or VS Code + Copilot.

1. Unzip the kit and open the `starter/` folder in your AI tool (Cursor: File > Open Folder; Claude Code: `cd starter && claude`).

2. Install and run:

```
npm install
npm run dev
```

A browser tab opens with a tiny platformer. Play it: arrows to move, Space to jump.

3. Make your first save point:

```
git init
git add -A
git commit -m "starter works"
```

4. Plan your game. Fill in the one-page `GDD.md`. Keep v1 tiny.

5. Pick your genre in `PROMPTS.md` and paste Prompt 0, then P1. Test. Commit. Repeat. (Finished v1 and want more content? See `GROW-YOUR-GAME.md`.)

6. When something breaks, open `FIX-LOOP.md`. Three tries, then roll back.

7. Ship it with `PUBLISH.md`: itch.io, GitHub Pages, or Netlify.

Using Lovable or Bolt? These run in the browser instead of opening a local folder. Bolt can import a project from GitHub; Lovable is built around its own React template, so for Lovable the smoothest route is to keep the game code in your own GitHub repo and use an editor (Cursor, Claude Code, Windsurf) for game work. In either case, paste `AGENTS.md` as your first message so the AI follows the rules.

What's in the kit

File	Use it for
`starter/`	Working Phaser 3 + Vite game with menu, game, and game-over scenes, art drawn in code, and a bundled OFL font (`starter/CREDITS.md`)
`starter/AGENTS.md` , `CLAUDE.md` , `.cursorrules`	Rules your AI reads automatically: architecture + "small steps"
`THEME.md`	The kit's visual theme: palette, color meanings, font, UI style, and how each genre looks in it
`GDD.md`	One-page game design doc template
`PROMPTS.md`	6 genre prompt sequences with test checkpoints
`GROW-YOUR-GAME.md`	Adding levels, waves, enemies, weapons, items, and modes after v1: data tables, the one-prompt rule, balance tips, a playtest loop, and example prompts for each genre
`FIX-LOOP.md`	Debugging and rollback system
`PUBLISH.md`	Publishing to itch.io, GitHub Pages, Netlify
`LICENSE.md`	What you can and can't do with the kit

The 4 rules that make vibe-coded games actually ship

1. **One prompt = one feature.** Never ask for five things at once.
2. **Test after every change.** Play it. Look at the console (F12).
3. **Commit every working step.** Git is your save game.
4. **Three failed fixes = roll back.** Then ask for a smaller step.

Browser Game Starter Kit by Prompt Forge Kits · promptforgekits.com · X @promptforgekits
Support: promptforgekits.com or email **[TODO: support email]**

One-Page Game Design Doc

> Fill this in **before** you prompt your AI. Paste it into your first chat so the AI knows what you're building. Keep it to one page. If it doesn't fit, your game is too big for v1.

Working title: Genre: (platformer / roguelite / idle / runner / puzzle / tower defense / other) **One-sentence pitch:** "It's a ___ where you ___ to ___." **Target session length:** (e.g. 2-minute runs, 10-minute levels) **Platform:** Browser (desktop keyboard / mouse / touch?)

1. Core loop (the 10 seconds you repeat)

1. Player does:
2. Game responds with:
3. Which leads to:

2. Controls

Input	Action

3. Win / lose conditions

- **Win:**
- **Lose:**
- **Score / progress measured by:**

4. Entities (keep it under 6 for v1)

Name	Looks like (placeholder)	Behavior
Player	teal square	

5. Progression and difficulty

How does it get harder or more interesting over time? (speed ramps, new enemy types, more levels, upgrades)

6. Feel and juice (add after the game works)

- [ ] Screen shake on hit - [ ] Particles on collect - [ ] Sound effects - [ ] Tweened UI - [ ] Hit-stop

7. Scope fence

- **v1 MUST have (max 5):**
- **v1 will NOT have (write these down so you don't add them):**
- **Maybe in v2:**

8. Art and audio plan

Start with shapes drawn in code in the kit's style (THEME.md). Final art source: (self-made / CC0 pack / commissioned). Audio source:

9. Done means

"v1 is done when a stranger can open the itch.io link, understand what to do within 10 seconds, and play a full run from menu to game over."

Ship date: ____ (pick one within 2 weeks)

Visual Theme: "Chunky Flat Toy"

One look for every game you build with this kit: flat shapes, a thick dark outline, one highlight per shape, no gradients on actors, and a fixed 16-color palette. Backgrounds are muted and low-contrast; characters are bright and outlined, so they always stand out. Everything is drawn in code with Phaser Graphics, so no image files are needed and there is nothing to license. The starter platformer already uses it (`starter/src/textures.js`).

1. Palette (16 colors)

Defined as `PALETTE` in `src/config.js`. Use these names in code, never new hex values.

Name	Hex	Role
`ink`	`#1b1f2e`	Every outline, dark text, shadows
`slateDark`	`#2e3550`	Panels, terrain body, ground fill
`slate`	`#4a5373`	Terrain, panel highlight, disabled buttons
`slateLight`	`#8a93ad`	Far hills, poles, inactive UI
`skyMuted`	`#a9c4d4`	Sky top, far background
`cloud`	`#e9edf5`	Sky bottom, clouds, light panels
`white`	`#ffffff`	Text, highlights, **hit flash**
`oliveDark`	`#55613a`	Terrain shading
`olive`	`#7d8a52`	Grass, near hills, trees
`oliveLight`	`#a9b47a`	Grass highlight, near hills
`red`	`#ef4f4f`	**Danger**: enemies, spikes, damage, game over
`gold`	`#ffc93c`	**Money / reward**: coins, score, prices, goal
`cyan`	`#3ec9e6`	**The player** and player projectiles
`blue`	`#4d7cff`	**Friendly / slowed**: allies, frost, slow rings
`green`	`#5bd16b`	**Heal / poison**: health pickups, poison, "you win"
`purple`	`#a45ee5`	**Magic / rare**: power-ups, rare loot, special abilities

Fixed color meanings (same in every genre): red = danger, gold = money, cyan/blue = player/friendly/slowed, green = heal/poison, purple = magic/rare, white flash = hit. Terrain stays olive and slate. A tower, weapon, or effect uses the color of its damage type, and so do its projectiles.

2. Shape rules

- Outline: `OUTLINE` = 3 px of `ink` around every actor, pickup, and UI panel. Backgrounds get no outline.
- One highlight per shape: a small white blob at 40-60% alpha near the top-left.
- Characters get a big head or a big single shape, with large white eyes and ink pupils.
- Every enemy type gets a distinct shape and speed, not just a color: circle (basic), triangle (fast), square (tanky), diamond (flying), hexagon (boss).

- Backgrounds: two parallax layers of soft hills in the muted colors (`skyMuted` , `slateLight` , `olive` , `oliveLight`), sky gradient `skyMuted` to `cloud` .
- Health bars appear only after damage. Status effects are a colored ring (blue slow, green poison, purple magic).

3. Font

Lilita One (SIL Open Font License 1.1), bundled at `starter/public/assets/fonts/LilitaOne-Regular.ttf` with `OFL.txt` , loaded in `BootScene` via the `FontFace` API. Create all text with `addText(scene, x, y, text, size, color)` from `src/ui.js` : it adds the ink outline and a 3 px drop shadow. Sizes: 56 titles, 22-26 HUD and buttons, 16-20 body. Text stays short.

4. UI components (`src/ui.js`)

Component	Look	Code
Text	Lilita One, white or a role color, ink outline (size/5, min 4 px) + ink shadow	`addText()`
Panel	`slateDark` rounded rect (r 18), 4 px ink outline, solid ink drop shadow 6 px, `slate` highlight bar	`addPanel(scene, w, h, x?, y?)`
Background	Sky gradient + `bg-far` (scroll 0.15) + `bg-near` (scroll 0.4)	`createParallax()` , `updateParallax()`
HUD	Top-left: icon + number (gold for money), size 26, `setScrollFactor(0)` . Top-right: pause/speed.	GameScene `createHud()`
Button (to add)	Small `addPanel` + `addText` ; hover = gold text; only the button is interactive	build from the two helpers
Particles	8 px white `spark` tinted by role: gold collect, red hurt, gold win, green heal, purple magic	GameScene `burst()`
Motion	Sine bobs (coins 600 ms, titles 1200 ms), pulsing prompts, title pop with `Back.out` , 200 ms shake on death, 300 ms fade-in	scenes

5. The 6 genres in this style

All drawn with Graphics in `textures.js` ; reuse the starter's draw functions where noted.

Genre	Player	Enemies / obstacles	World and UI
Platformer (starter)	Cyan blob `drawPlayer`	Red circle walkers, red spikes	Olive/slate tiles, gold coins and flag
Endless runner	Same cyan blob, run anim	Red spikes (ground), red diamonds (flying), tall slate walls	Scrolling `bg-far` / `bg-near`, distance in white, coins gold
Top-down roguelite	Cyan blob seen from above, cyan bullets	Red circle / triangle (dasher) / square (brute), red bullets	Slate floor tiles with olive edges; upgrade cards as panels with purple border for rare
Idle / clicker	Big gold coin as the click target	n/a	Shop rows as panels, prices in gold, disabled = `slate`; purple for prestige
Puzzle (Sokoban)	Cyan blob	n/a	Slate floor, ink walls, gold-outlined targets, olive crates turning green on target
Tower defense	Towers as outlined shapes colored by damage type: cyan bolt, blue frost, purple arc (use `gold` for a splash/mortar tower)	Shape-coded red enemies per section 2	Olive ground, `cloud` path, dashed-circle build pads, range circle on hover; lives red, gold gold, wave white

Store pages use the same look: `ink` to `slateDark` background, Lilita One headline in white and gold with ink outline, chunky pill chips with ink borders, game screenshots in an ink-framed card, big code-drawn sprites rendered at 6x.

6. Sources and licenses

Item	Source	License
All art	Drawn in code, `starter/src/textures.js`	Part of the starter code (`LICENSE.md`)
Lilita One font	Juan Montoreano, via Google Fonts	SIL OFL 1.1 (`starter/public/assets/fonts/OFL.txt`)

Want image files later? Kenney's CC0 packs (kenney.nl) are safe to use; pick flat, outlined ones so they match, and list them in `starter/CREDITS.md`.

Prompt Playbook

Copy-paste prompt sequences for six genres. Each genre has a kickoff prompt (Prompt 0) plus 10 numbered prompts (Tower Defense has 13, plus 2 optional). Each sequence turns the starter into a different game, one small, testable step at a time.

How to use this playbook

1. Open the `starter/` folder in your AI tool (Cursor, Claude Code, Windsurf, Copilot). Most tools pick up the matching rules file (`.cursorrules` , `CLAUDE.md` , or `AGENTS.md`) automatically; if yours doesn't, paste `AGENTS.md` as your first message. *Lovable / Bolt users:* see the note in the Quick Start, and paste `AGENTS.md` as your first message.
2. Run `npm run dev` and keep the game open in a browser tab.
3. Paste **one prompt at a time**. Do not combine prompts.
4. After each prompt, do the **Test** checkpoint. If it passes, commit: `git add -A && git commit -m "step N works"`
5. If it fails, use `FIX-LOOP.md` . Three tries, then roll back.

Text in `<angle brackets>` is yours to customize. Prompt 0 is the same for every genre:

> **Prompt 0 (kickoff):**
>
> ```
> Read AGENTS.md and the files in src/. Summarize the architecture in 5 bullets. Then here is my
> game design doc:
> <paste your filled-in GDD.md>
> Don't write code yet. Tell me which existing files we'll keep, change, or replace for this game.
> ```
>
> **Test:** The summary mentions the 3 scenes and config.js. The plan sounds right to you.

---

1. Platformer

Builds on the starter as-is. Goal: a 3-level platformer with good jump feel.

P1. Better jump feel

```
Improve jump feel in GameScene: add coyote time (100 ms after leaving a ledge you can still jump) and
jump buffering (pressing jump up to 100 ms before landing still jumps). Put both values in config.js.
```

Test: Run off a ledge and press jump slightly late. You still jump.

P2. Variable jump height

```
Make jump height depend on how long jump is held: if the jump key is released while moving upward,
cut upward velocity in half. Add the multiplier to config.js.
```

Test: Tap = small hop, hold = full jump.

P3. Moving platforms

```
Add moving platforms that slide horizontally between two points. The player must ride along when standing on one. Add two of them to the current level, defined in a MOVING_PLATFORMS array at the top of GameScene.js. Show a name above the first one the first time it is on screen ("Moving platform: ride it"). Keep the strings in an INFO object in config.js and draw them with the addTag / addBanner / addLegend / addTooltip helpers from ui.js.
```

Test: Stand on a moving platform. You move with it and don't slide off. The first one is labeled.

P4. Stomp enemies

```
Let the player defeat an enemy by landing on it from above (player falling and above the enemy). The enemy disappears, the player bounces up, and score increases by 25. Touching it from the side still ends the game. Name the first enemy on screen ("Enemy: stomp it +25", text in INFO in config.js).
```

Test: Jump on a spike from above = it vanishes and you bounce. Walk into one = game over. The enemy has a name above it.

P5. Lives and respawn

```
Give the player 3 lives. Losing a life respawns them at the level start (or last checkpoint later) instead of ending the game. Show lives in the HUD with a label ("Lives: 3"). Game over only at 0 lives.
```

Test: Die twice: you respawn and the HUD shows 1 life left. Die a third time: game over screen.

P6. Checkpoints

```
Add a checkpoint object (a small flag drawn in textures.js). Touching it saves it as the respawn point and tints it green. Add one to the middle of the level and name it the first time it is on screen ("Checkpoint: respawn here").
```

Test: Touch the checkpoint, die, respawn there.

P7. Multiple levels

```
Move level data (platforms, coins, enemies, goal, checkpoints) out of GameScene into src/levels.js as an array of level objects. Reaching the goal loads the next level, keeping score and lives. After the last level, go to GameOverScene with won: true. Create 3 levels of increasing difficulty. Show a one-line goal banner ("Reach the flag!", text in INFO in config.js) when each level starts. Restarting from GameOverScene or the menu must start a fresh run at level 1 with score 0 and full lives: always pass that data explicitly, e.g. `this.scene.start('GameScene', { levelIndex: 0, score: 0, lives: 3 })`, because scene.start() without data reuses the previous run's data.
```

Test: Beat level 1, level 2 loads with your score kept. Beating level 3 shows YOU WIN. Press Space there: you restart at level 1 with score 0.

P8. Level indicator and transitions

```
Show "Level N" in the HUD, and a short fade-out/fade-in camera transition between levels.
```

Test: The screen fades between levels and the HUD label updates.

P9. Juice

```
Add juice: small squash-and-stretch tween when the player lands, particle burst when collecting a coin, and a short camera shake when losing a life. Keep each effect subtle.
```

Test: All three effects visible; game still runs smoothly.

P10. Sound

```
Add simple sound effects for jump, coin, stomp, and death using the Web Audio API to generate short beeps (no audio files). Put them in src/sfx.js. Add an M key to mute during gameplay (GameScene only; GameOverScene keeps M for "menu"), saved in localStorage.
```

Test: You hear each sound; M toggles mute and survives a page refresh.

Ship check: Play all levels start to finish. Then follow PUBLISH.md.

---

2. Top-Down Roguelite

Replaces the platformer with a top-down arena: rooms, enemies, upgrades between rooms.

P1. Top-down movement

```
Convert GameScene to top-down: turn gravity off for this scene, remove platforms, coins, enemies and goal. The player moves in 8 directions with WASD/arrows at constant speed (normalize diagonals). Add a rectangular room with walls around the screen edges the player collides with. Keep the menu and game over scenes, but change MenuScene's title to my game name: <name> (not "TINY PLATFORMER") and its controls line to the new controls.
```

Test: You move in all 8 directions, diagonals aren't faster, walls block you.

P2. Shooting

```
Let the player shoot bullets toward the mouse pointer on click (hold to auto-fire at a fire rate from config.js). Bullets are small circles that are destroyed on hitting walls or after 1 second.
```

Test: Click shoots toward the cursor; holding fires repeatedly; bullets don't pile up forever.

P3. Chaser enemy

```
Add one enemy (spawned at a fixed spot away from the player for now) that walks toward the player. Bullets damage it (3 HP); it flashes white when hit and dies at 0 HP. Touching the player deals 1 damage. Player has 5 HP shown in the HUD; at 0, go to GameOverScene. Give the player 1 second of invulnerability (blinking) after being hit. Show HP as "HP: 5/5" and name the first enemy on screen ("Chaser: touch = -1 HP"). Keep the strings in an INFO object in config.js and draw them with the addTag / addBanner / addLegend / addTooltip helpers from ui.js.
```

Test: Shoot an enemy 3 times to kill it. Getting touched lowers HP once, not every frame. The enemy is named and the HUD says "HP: 5/5".

P4. Waves in a room

```
Each room spawns a wave of 4 enemies (the next prompt makes this grow) at random positions at least 200px away from the player. When all are dead, show "Room cleared!" and open a door (a gap in the right wall). In room 1 show a one-line goal banner ("Kill every enemy, then walk through the gold door") and label the door ("Door: opens when room is clear").
```

Test: Kill all enemies, door opens; the door is closed while enemies are alive.

P5. Room progression

```
Walking through the open door loads a new room with a bigger wave (room number * 2 + 2 enemies). The player's current HP carries over to the next room; pass it in the scene data. Show the room number in the HUD with labels ("Room: 2   Cleared: 1"). Score = rooms cleared. Starting a new run from GameOverScene or the menu must pass fresh data explicitly (room 1, full HP), because scene.start() without data reuses the old data.
```

Test: Room 2 has more enemies than room 1; the HUD counter goes up.

P6. Second enemy type

```
Add a shooter enemy that keeps its distance and fires slow bullets at the player every 2 seconds. Mix it into waves starting from room 3. Name the first shooter on screen ("Shooter: dodge its shots").
```

Test: From room 3, some enemies shoot at you; their bullets damage you.

P7. Upgrade choice

```
After clearing a room, pause and show 3 random upgrade cards (e.g. +20% fire rate, +1 max HP and heal, +15% move speed, bullets pierce one enemy, +1 damage). Each card shows a short name and a one-line description ("Rapid Fire: Shoot 20% faster"), both kept in an UPGRADE_TEXT object in config.js. Clicking one applies it and resumes. While the cards are open, set a `choosing` flag and skip player shooting so the card click does not also fire a bullet. Put upgrade definitions in src/upgrades.js. Show "Upgrades: N" in the HUD, with a hover tooltip listing the upgrades taken. Keep the player's stats (max HP, fire rate, speed, damage, pierce, upgrades taken) in one `stats` object that is passed to the next room in the scene data, so upgrades persist between rooms; a new run starts from the default stats.
```

Test: Every card has a name and a one-line description. Pick "+fire rate" and you visibly shoot faster; hovering "Upgrades: 1" lists it.

P8. Run summary

```
Pass rooms cleared and upgrades taken to GameOverScene and display them. Save best rooms cleared as the high score.
```

Test: Die; the summary lists your upgrades and rooms reached.

P9. Boss room

```
Every 5th room is a boss room: one large enemy with 40 HP and a health bar at the top of the screen, which alternates between chasing and firing a ring of 8 bullets. Label the health bar "BOSS" and name the boss the first time it is on screen ("BOSS: 40 HP, dodge the ring").
```

Test: Room 5 has the boss with a health bar; beating it opens the door.

P10. Juice

```
Add juice: screen shake when the player is hit, small knockback on enemies when shot (read the bullet's velocity before destroying the bullet), particle burst on enemy death, and a brief hit-stop (50 ms pause) on boss hits.
```

Test: Combat feels punchier; no slowdowns over a few rooms.

Ship check: Do a full run to room 6+ without errors.

---

3. Idle / Clicker

Replaces gameplay with a clicker economy. No physics needed.

P1. Click to earn

```
Replace GameScene with an idle clicker: a big circle in the center that gives +1 coin when clicked, with a small scale "bounce" tween on click. Show the coin total at the top, labeled ("Coins: 0"), and a bobbing "Click the coin!" hint above the circle until the first click. Remove the platformer code and the game over trigger for now.
```

Test: Each click adds exactly 1 and the circle bounces.

P2. Floating numbers

```
When the circle is clicked, spawn a "+1" text at the pointer that floats up and fades out over 600 ms, then is destroyed.
```

Test: Spam-click: the numbers float and vanish; the game doesn't slow down.

P3. First upgrade

```
Add a shop panel on the right with one upgrade: "Better Click" (+1 coin per click). Cost starts at 10 and multiplies by 1.15 per purchase. Button is greyed out when unaffordable. Put upgrade data in src/economy.js. Each shop row shows the name, a one-line description built from its numbers ("+1 coin per click"), the cost and how many you own; hovering a row shows a tooltip (what you own, what it gives now). Keep the strings in an INFO object in config.js and draw them with the addTag / addBanner / addLegend / addTooltip helpers from ui.js.
```

Test: Buy it at 10 coins; clicks now give 2; next cost is 11 or 12.

P4. Passive income

```
Add generator upgrades that produce coins per second automatically: "Helper" (0.5/s, cost 25), "Machine" (4/s, cost 250), "Factory" (30/s, cost 3000), each with 1.15x cost scaling. Show coins per second in the HUD. Update income with delta time, not per frame. Label the HUD number ("2.0 coins/s") and give each generator a one-line description ("+0.5 coins/s each").
```

Test: Buy a Helper; coins rise ~0.5/s even without clicking.

P5. Number formatting

```
Format large numbers: plain with commas below 10,000 (1,234), then 12.3K, 4.56M, 7.89B, etc. Use it everywhere numbers are shown. Put the formatter in src/format.js.
```

Test: Ask the AI for a temporary debug key that adds 5,000,000 coins; press it and the display shows "5.00M".

P6. Save and load

```
Save the full game state to localStorage every 5 seconds and on page close. Load it on startup. Add a "Reset save" button in the menu with a confirm step. The starter menu starts the game on a click anywhere, so make only the start text (and the reset button) clickable, or clicking Reset will also start the game.
```

Test: Earn coins, refresh the page; your progress is still there.

P7. Offline earnings

```
When the game loads, calculate time since the last save (cap at 8 hours) and grant coins per second * seconds away * 0.5. Show a popup "While you were away you earned X".
```

Test: Note your coins, close the tab for 1 minute, reopen; you get a popup with ~30x your /s.

P8. Milestones

```
Add milestones: owning 10, 25, 50 of a generator doubles its output. Show the next milestone on each shop button ("next x2 at 10") and explain it in that row's tooltip ("Owning 10 doubles its output").
```

Test: Reach 10 Helpers and your /s jumps for Helpers.

P9. Prestige

```
Track total coins earned (every coin gained from clicks, generators and offline earnings; spending does not lower it) and save it. Add a Prestige button that unlocks at 1M total coins earned. Prestiging resets coins and upgrades but grants "Stars" = floor(sqrt(totalEarned / 1M)); each Star gives +10% to all income (clicks and generators) permanently. Explain it in a confirm popup. Also show the goal in the shop footer ("Prestige: 120K / 1M earned", then "Prestige is ready!") with a tooltip that explains Stars, and label the HUD "Stars: N".
```

Test: Use the debug key to pass 1M total earned, then prestige; you keep Stars and income is boosted.

P10. Polish

```
Add polish: generators pulse when they produce, the shop button flashes when a new upgrade becomes affordable, and a soft click sound via Web Audio. Change MenuScene's title to my game name: <name>. Make the menu say what to do: a one-line goal plus an icon + name legend for the big coin, coins and Stars.
```

Test: It feels alive while idle. Remove any debug keys before publishing.

Ship check: Leave it running 5 minutes; refresh; offline popup and save both work.

4. Endless Runner

Auto-running, jump-to-survive, score = distance.

P1. Auto-run setup

```
Convert GameScene to an endless runner: the player stays at x=150 and the ground scrolls left (use a long TileSprite floor with tilePositionX increasing). Remove the level arrays. Space/Up/click jumps. Keep gravity. Change MenuScene's title and controls line, and add a one-line goal ("Run as far as you can. Hit nothing!").
```

Test: The ground scrolls, the player runs in place and can jump.

P2. Obstacles

```
Spawn obstacles (spikes from textures.js) off the right edge at random intervals of 1-2.2 seconds; they move left at the world speed and are destroyed off-screen. Touching one ends the game. Name the first spike on screen ("Spike: jump!") with an addTag label. Keep the strings in an INFO object in config.js and draw them with the addTag / addBanner / addLegend / addTooltip helpers from ui.js.
```

Test: Obstacles come steadily; hitting one goes to game over. Check with console that old obstacles get destroyed.

P3. Distance score

```
Score = distance in meters (world speed * time / 50). Show it in the HUD as "Distance: N m"; pass it to GameOverScene and keep the high score logic.
```

Test: Score climbs smoothly; game over shows it and Best updates.

P4. Speed ramp

```
Increase world speed by 5% every 5 seconds up to a max in config.js. Spawn intervals shrink slightly as speed rises so spacing stays fair.
```

Test: After 30 seconds it's noticeably faster but still jumpable.

P5. Double jump

```
Allow one extra jump in the air, triggered on key press (a new press, e.g. Phaser.Input.Keyboard.JustDown or a keydown event), not while the key is held with isDown, or holding jump uses up the double jump instantly. Reset when landing. Show a small puff of particles on the second jump. Add "Jump again in the air to double jump" as a second line under the menu controls.
```

Test: You can jump twice, not three times.

P6. Varied obstacles

```
Add two more obstacle types: a tall wall (about 120 px: just taller than a single jump, which reaches about 117 px with the starter's values, so it needs a double jump; check the numbers in config.js) and a flying block at head height (must stay low / not jump). Pick randomly with weights, introducing the flying block after 200 m. Name each type the first time it appears ("Wall: double jump!", "Flyer: stay low!") and list all obstacles in an icon + name legend on the menu.
```

Test: All three types appear and each is passable.

P7. Coins and pickups

```
Spawn coin lines (3-5 coins in an arc) between obstacles. Coins add +5 to a separate coin counter. Every 10 coins grants a 3-second shield that absorbs one hit (shown as a circle around the player). Show the counter as "Coins: 3   (shield in 7)" and "SHIELD ON" while it lasts, and name the first coin arc ("Coin: 10 = shield").
```

Test: Collect 10 coins, get a shield, hit an obstacle and survive once.

P8. Parallax background

```
Add 2 parallax background layers (distant hills and clouds made from shapes) that scroll at 20% and 50% of world speed.
```

Test: Layers move at different speeds, giving depth.

P9. Mobile controls

```
Make it mobile-friendly: tap anywhere to jump, and make sure the canvas scales to fit portrait and landscape phones. Show "Tap to jump" on the menu when on a touch device.
```

Test: Open dev tools device mode (Ctrl+Shift+M) and play with clicks as taps.

P10. Juice

```
Add juice: dust particles while running, screen shake on death, a slow-motion (timeScale 0.3 for 400 ms) moment before the game over screen, and Web Audio beeps for jump and coin.
```

Test: Death feels dramatic, not abrupt.

Ship check: Play 5 runs; the difficulty curve feels fair.

---

5. Puzzle (Grid / Sokoban-style)

Grid-based, turn-based, with handcrafted levels.

P1. Grid and player

```
Convert GameScene to a grid puzzle: don't use physics bodies in this scene (set its gravity to 0). Draw a 10x8 grid of 48px tiles centered on screen. The player occupies one tile and moves one tile per key press (arrows/WASD) with a 100 ms tween. No diagonal moves.
```

Test: One key press = exactly one tile, even if you hold the key.

P2. Levels as text

```
Define levels as arrays of strings in src/levels.js using: '#' wall, '.' floor, 'P' player, 'B' box, 'T' target. Parse and render level 1, centered on screen (levels can be smaller than 10x8). The player can't walk into walls. On level 1 show a name above the player, a crate and a target ("You", "Crate: push it", "Target: crate goes here").
```

Test: Level 1 renders matching the strings; walls block movement.

P3. Pushing boxes

```
The player pushes a box when walking into it if the tile behind the box is floor or target. Boxes can't push other boxes or go into walls.
```

Test: Push a box; try pushing it into a wall and into another box: both blocked.

P4. Win condition

```
When every target has a box on it, show "Solved!" and load the next level after 1 second. Boxes on targets turn green. After the last level, go to GameOverScene with won: true. Add a small second level so this can be tested. Update the menu with a one-line goal ("Push every crate onto a target") and an icon + name legend: crate, target, green crate = done, you. Pass the level in the scene data (`{ levelIndex }`), and make the restart from GameOverScene and the menu pass `{ levelIndex: 0 }` explicitly, because scene.start() without data reuses the previous data.
```

Test: Solve level 1 and level 2 loads. After the last level, play again starts at level 1.

P5. Undo and restart

```
Add Undo (Z) that reverses the last move including box pushes, using a history stack of states. Add Restart (R) to reset the current level.
```

Test: Push a box, press Z: both player and box go back. R resets.

P6. Move counter

```
Count moves per level and show them in the HUD with labels ("Moves: 12   Best: 9"; Undo takes one move back). Save the best move count per level in localStorage and show it.
```

Test: Solve a level; the best moves saves and shows on replay.

P7. Level select

```
Add a LevelSelectScene reachable from the menu with a button and the L key (the starter menu starts the game on any click, so make only the start text and this button clickable): a grid of level buttons, locked until the previous level is solved (progress saved in localStorage). Show the best moves on solved levels. Locked buttons say "LOCKED" and a hint line says how to unlock the next one.
```

Test: Only level 1 is unlocked at first; solving it unlocks level 2 after a refresh.

P8. More levels

```
Design 8 more levels that teach one idea each, from very easy to tricky: corners trap boxes, two boxes, pushing order matters, etc. Make sure every level is solvable; list the solution move sequence for each in a comment. Write a small Node script (scripts/check-levels.mjs, not part of the game) that replays each listed solution against the level and reports any that don't end solved. Give every level a one-line hint shown under the HUD (a levelHints array in INFO in config.js), e.g. "Corners trap crates".
```

Test: Run `node scripts/check-levels.mjs`, then play every level yourself. Remove any that are impossible or boring.

P9. Touch controls

```
Add swipe controls for mobile (swipe direction = move) and on-screen Undo/Restart buttons labeled with their keys ("UNDO (Z)", "RESTART (R)").
```

Test: In dev tools device mode, drag to move.

P10. Juice

```
Add juice: boxes squash slightly when pushed, a sparkle burst when a box lands on a target, a camera shake + "bonk" sound when bumping a wall, and a happy arpeggio on level solved (Web Audio).
```

Test: Feedback for every action; nothing slows down undo.

Ship check: Solve all levels from a fresh save (clear localStorage).

---

6. Tower Defense: "Core Keepers"

Four towers with distinct roles, six shape-coded enemies, 20 waves, and two signature mechanics: **Recoverable Cores** (an enemy that reaches your vault steals one of your 8 cores and walks it back to the spawn gate; kill it and the core floats home) and **Overcharge** (3x fire rate on one tower, hotkey Q). 13 prompts, plus 2 optional ones. Every gameplay number lives in **one file,** `src/settings.js`, so you balance the game by editing numbers, not code. Each prompt quotes the numbers it needs; the values shown are the ones our bot-tested build shipped with.

P1. Setup and the settings file

```
Turn this starter into a tower defense game called "Core Keepers".
- config.js: GAME_WIDTH 960, GAME_HEIGHT 540, gravity 0. Keep PALETTE, OUTLINE, FONT.
- Remove the platformer level, player, coins, and parallax from GameScene (keep BootScene, MenuScene, GameOverScene, ui.js, textures.js). GameScene shows an empty olive screen for now.
- Create src/settings.js. It is the ONE place for every gameplay number from now on (towers, enemies, waves, economy, abilities); other files import from it and never hard-code a stat. Start it with:
export const ECONOMY = { startGold: 100, startCores: 8 };
- textures.js: add and export drawShape(g, shape, x, y, size, fill) that draws a 'circle', 'triangle', 'square', 'diamond' or 'hexagon' of radius size in color fill, with the 3 px ink OUTLINE and one small white highlight at 45% alpha near the top-left (see THEME.md).
- MenuScene: title "CORE KEEPERS", instruction line "Click pads to build   Q: Overcharge". Replace the platformer sprites with three drawShape icons (cyan triangle, red circle, gold hexagon).
Do not change unrelated code; list what you changed.
```

Test: the menu says CORE KEEPERS at 960x540; clicking starts an olive screen; the console has no errors.

P2. Path, vault, and build pads

```
Create src/map.js exporting:
- PATH: waypoints from a spawn gate on the left edge to a core vault near the right edge, with 6-8 turns. Keep the road 40 px from the screen edges and out of the top 56 px (HUD).
- PADS: 10 build-pad centers, each 50-70 px from the road center and at least 60 px from each other, never on the road.
- SPAWN and VAULT (the first and last waypoint).
Draw in GameScene, below everything else: olive ground with a few muted oliveLight grass tufts; the road as a 48 px slateLight band with a 40 px cloud band on top (so it has a soft edge); each pad as a dashed slateLight circle (radius 24) on a slightly darker olive disc; the spawn as an ink/red gate; the vault as a big gold hexagon (drawShape) with a ring of small gold core dots (one per core, ECONOMY.startCores) that we will update later.
Do not change unrelated code; list what you changed.
```

Test: one clear road snakes from the gate to the vault; 10 dashed pads sit beside it; no pad touches the road.

P3. First enemy and the game clock

```
Add an Enemy class in src/enemy.js and ENEMIES in settings.js, starting with:
grunt: { hp: 40, speed: 55, size: 14, shape: 'circle' }
- Enemies move by distance along PATH (no physics bodies). Store dist and compute x,y from it.
- All gameplay logic runs in one method GameScene.tick(dt) where dt is milliseconds. update(time, delta) calls this.tick(delta * this.speed) unless this.paused (this.speed = 1 for now). Keep a scaled clock this.gameTime += dt and use it for every cooldown and spawn timer. Do NOT use this.time.now, this.time.delayedCall, or tweens for gameplay (they ignore our speed and pause); tweens are fine for pure visuals.
- Draw enemies with drawShape in red, size from settings, with two white eyes and ink pupils. Each enemy has a health bar that stays hidden until it first takes damage.
- For testing only: spawn a grunt every 1.5 s of game time. An enemy that reaches the vault is removed.
Do not change unrelated code; list what you changed.
```

Test: grunts walk the road smoothly from the gate to the vault and disappear there.

P4. Radial build menu

```
Add TOWERS to settings.js (more stats come later):
bolt: { name: 'Bolt', cost: 50, color: 'cyan', shape: 'triangle' }
mortar: { name: 'Mortar', cost: 100, color: 'gold', shape: 'circle' }
frost: { name: 'Frost', cost: 75, color: 'blue', shape: 'diamond' }
arc: { name: 'Arc', cost: 125, color: 'purple', shape: 'hexagon' }
and ECONOMY.costGrowth: 0.10. Also give every tower two text strings in settings.js (UI text lives there, not in code): role (one short line) and desc (one line), e.g. bolt role 'Fast single-target, hits flyers'; mortar 'Splash damage, ground only'; frost 'Slows enemies in range'; arc 'Chain damage, pierces armor'. Keep every line under 8 words where you can.
- In textures.js add drawTowerIcon(g, type, x, y, size, aim) so every tower reads at a glance in the Chunky Flat Toy style (same 3 px ink outline, one flat color per tower, drawn in code): Bolt = cyan square base, round turret dome with a gold lightning mark and a long barrel (aim turns the barrel); Mortar = gold squat wheeled body, fat slate tube pointing up, round dark shell on top; Frost = blue six-armed snowflake with white tips inside a blue ring; Arc = purple tesla coil (stacked tapered mast, glowing ball on top, two gold spark zigzags). Use this one function for the menu, the map, and the info card.
- Clicking an empty pad opens a radial menu: 4 round icons (radius 26, slateDark with ink outline, drawTowerIcon size 15) placed around the pad, with the tower name and price under each ("Bolt 50") in gold (affordable) or slate (not).
- Hovering or selecting an icon shows an info card (panel in a free screen corner, away from the pad) with: name and price, the role line, the desc line, and stats read from settings.js (the tower icon again, Damage, Rate in shots/s, Range, plus Splash or Chains where they exist; Frost shows Aura and Slow). The selected icon's card stays open. Put the hint "Click again to build" at the bottom of the card (UI_TEXT in settings.js).
- First click on an icon selects it (gold ring + the text "Click again"); a second click on the same icon builds the tower on that pad, takes the gold, and closes the menu. Clicking anywhere else closes the menu.
- Price = base cost x (1 + costGrowth x towersBuilt), rounded to the nearest 5. towersBuilt counts every tower ever built this run.
- Tower look: drawTowerIcon, size 18, on the pad.
- HUD top-left: a gold coin icon and the gold number (size 26). Clicks on UI objects must not also count as map clicks (check the objects under the pointer before handling a map click).
Do not change unrelated code; list what you changed.
```

Test: hover each icon: the card shows its name, role and stats; the four icons look different even when small. With 100 gold, build a Bolt (50). The next Bolt shows 55. Click empty grass: the menu closes and nothing is built.

P5. Bolt and Mortar shoot

```
Add combat stats to settings.js:
bolt: damage 8, rate 0.4 (seconds between shots), range 130, projectileSpeed 520, hitsFlyers true
mortar: damage 20, rate 1.5, range 150, splash 40, projectileSpeed 260, hitsFlyers false
- Each tower has targetMode 'first' (the enemy furthest along the path, default) or 'nearest'. It only looks at enemies inside its range.
- Bolt fires a small cyan dart that homes on its target and vanishes if the target dies. Mortar lobs a gold shell to where the target was when it fired; on landing it damages every ground enemy within splash.
- Cooldowns use this.gameTime. Projectiles move in tick(dt).
- Killing an enemy gives its gold: add gold: 2 to grunt for now.
- Projectiles use their tower's color. The Bolt tower turns its barrel toward its target (redraw the icon with aim; do not rotate the whole graphics object, tier pips must stay put).
Do not change unrelated code; list what you changed.
```

Test: two Bolts kill grunts and the gold number rises. A Mortar hits a group of grunts with one shell.

P6. Frost and Arc, status rings

```
Add to settings.js:
frost: aura 2 (damage per second to every enemy in range), slow 0.4, range 90, hitsFlyers true
arc: damage 15, rate 1.0, range 120, chains 3, chainRange 80, ignoresArmor true, hitsFlyers true
- Frost has no projectile: enemies inside its range move 40% slower (several Frost towers do not stack; use the strongest slow) and show a blue ring around them while slowed. Draw a faint blue aura circle under the Frost tower plus a pulsing blue ring (a visual tween: scale up to 1.7 while fading out, repeating every 1.1 s).
- Arc instantly hits its target with a purple jagged lightning line, then jumps to the nearest enemy within chainRange that it has not hit yet, up to chains enemies in total. The line fades in 120 ms (visual tween is fine).
Do not change unrelated code; list what you changed.
```

Test: grunts visibly slow near Frost and get a blue ring; Arc's lightning hits 3 grunts in a row.

P7. The enemy roster

```
Complete ENEMIES in settings.js. Every enemy is red (red = danger) and reads by SHAPE and SIZE:
grunt  { hp: 40, speed: 55, size: 14, shape: 'circle', gold: 2 }
dasher { hp: 14, speed: 110, size: 10, shape: 'triangle', gold: 2 }
brute  { hp: 90, speed: 36, size: 19, shape: 'square', armor: 8, gold: 4 }
swarm  { hp: 7, speed: 70, size: 6, shape: 'circle', group: 8, gold: 1 }
glider { hp: 26, speed: 62, size: 13, shape: 'diamond', flying: true, gold: 3 }
boss   { hp: 600, shield: 300, speed: 28, size: 28, shape: 'hexagon', spawnOnHalf: { type: 'swarm',
count: 8 }, gold: 40 }
- armor: every hit loses armor damage (minimum 1 per hit) unless the tower ignoresArmor. Frost aura
ignores armor.
- shield: absorbs damage before hp; show it as a white ring that shrinks as the shield drops.
- group: one spawn creates that many, spaced 12 px apart along the path.
- flying: the glider ignores the road and flies in a straight line from SPAWN to VAULT, drawn 10 px
higher with an ink shadow ellipse under it. Towers with hitsFlyers false never target it.
- spawnOnHalf: when the boss first drops below 50% hp it releases those enemies at its position.
- Every enemy gets name and text (one line, 8 words or fewer, e.g. Brute: 'Armored. Use Arc.').
Hovering an enemy shows "Name: text" above it. Keep the eyes on every enemy.
- Add a debug key 1-6 that spawns one of each type in order (we remove it in P13).
Do not change unrelated code; list what you changed.
```

Test: press 1-6: six different silhouettes; Bolt hits gliders, Mortar ignores them; brutes shrug off Bolt (8 - 8, so the 1-damage minimum per hit) but not Arc.

P8. Wave manager

```
Remove the test spawner. Create src/waves.js that builds all 20 waves from WAVES in settings.js:
WAVES = { total: 20, countBase: 6, countPerWave: 2, countCap: 40, hpGrowth: 1.17, spawnGap: 0.8,
  unlock: { dasher: 4, swarm: 6, brute: 7, glider: 12 }, weights: { grunt: 4, dasher: 2, swarm: 1,
brute: 1, glider: 1 },
  bossWaves: [10, 20], breatherEvery: 5, breatherCount: 0.6, breatherGold: 1.5, countdown: 12,
clearBonus: 10, killGoldEvery: 3 }
- buildWave(n) returns a list of spawns. count = min(countCap, countBase + countPerWave x n), times
breatherCount on breather waves (5, 15; not boss waves). Split the count between unlocked types by
weight (round, grunts take the remainder), interleave them evenly, and multiply hp by hpGrowth^(n-1).
On a type's unlock wave its first spawn is that type alone with a 3 s gap after it, and the screen
shows a one-line tip of 8 words or fewer near the gate (e.g. "Dashers are fast. Frost slows them.").
Boss waves add one boss at the end.
- Kill gold = the enemy's gold + floor(n / killGoldEvery), times breatherGold on breather waves.
Clearing a wave pays clearBonus.
- Before wave 1 nothing spawns until the player clicks the "Next wave" chip. After a wave is cleared,
a countdown of countdown seconds starts; at 0 the next wave starts by itself.
- The "Next wave" chip sits next to the spawn gate: a small panel with the wave number and one mini
icon + count per enemy type in that wave. Hovering it lists the wave's enemies by name ("6 Grunt, 2
Dasher"). Clicking it during the countdown sends the wave early and pays +1 gold per whole second
left (show "+N" in gold).
- HUD top-left: add "Wave X/20" in white after the gold.
Do not change unrelated code; list what you changed.
```

Test: wave 1 is 8 grunts and starts only on click. The chip previews wave 2 and its countdown; sending early adds gold. Wave 4 shows the Dasher tip.

P9. Recoverable Cores

```
Replace "removed at the vault" with Recoverable Cores (CORES in settings.js: { floatSpeed: 30,
pickupRadius: 18 }; startCores stays in ECONOMY).
- The vault holds startCores cores. An enemy that reaches the vault takes one (if any are left),
turns around, and walks the path back to the spawn gate at the same speed (a glider flies straight
back). It carries a small gold core dot on top. If the vault is empty it just turns around. For
'first' targeting, an enemy walking back counts as further along than any enemy walking in (so
carriers are shot first).
- If an enemy carrying a core dies, the core drops there and floats back toward the vault along the
path at floatSpeed (gold dot with a soft pulse). Any enemy that is not carrying a core and touches a
floating core (pickupRadius) picks it up and carries it back to the gate.
- A core that reaches the spawn gate on an enemy is lost for good (flash the gate red). A floating
core that reaches the vault is returned.
- HUD top-left: the first item is "cores at home / cores not lost" with a gold hexagon icon, e.g.
"6/8". Update the vault's ring of dots to show how many are home.
- Game over (GameOverScene, won: false) when every core is lost for good. A wave only counts as
cleared when no enemies are left alive (floating cores may still be on their way home).
Do not change unrelated code; list what you changed.
```

Test: let a grunt through: the HUD shows 7/8 and it walks back with a core. Kill it on the way back: the core floats home and the HUD shows 8/8 again. Let one escape: 7/7.

P10. Upgrades, the fork, selling, targeting

```
Add UPGRADES to settings.js:
UPGRADES = { tierCost: 0.7, forkCost: 1.5, sellRefund: 0.7, tier1: { damage: 1.3 }, tier2: { range: 1.2 },
  forks: {
    bolt:   { A: { name: 'Railgun', text: 'Pierces a line', pierce: true, damage: 2.0, rate: 1.8 }, B: { name: 'Twin', text: 'Shoots 2 targets', targets: 2 } },
    mortar: { A: { name: 'Cluster', text: '3 mini blasts', cluster: 3 }, B: { name: 'Napalm', text: 'Burning ground', burn: { dps: 12, time: 3 } } },
    frost:  { A: { name: 'Shatter', text: 'Slowed take +25%', vulnerable: 0.25 }, B: { name: 'Freeze', text: 'Chance to stun', stun: { chance: 0.1, time: 1 } } },
    arc:    { A: { name: 'Overload', text: 'Breaks shields', shieldBreak: 3 }, B: { name: 'Storm', text: 'Chains to 6', chains: 6 } } } }
- Clicking a built tower selects it: show its range circle and a small panel beside it (addPanel + addText) with: the tower name and tier, its role line, a "Next:" line saying what the next upgrade does (strings in UI_TEXT in settings.js: 'Next: +30% damage', 'Next: +20% range', 'Next: choose A or B', 'Fully upgraded'), an Upgrade button (tier 1 and 2 cost tierCost x base cost; tier 1 = +30% damage, tier 2 = +20% range), then at tier 3 two fork buttons A / B with their name, 3-word text and price (forkCost x base cost; choosing one locks the other), a target-mode button (First / Nearest), and Sell (sellRefund x everything spent on it, Math.round). Buttons the player can't afford show the price in slate. Clicking elsewhere deselects.
- Fork rules: pierce = the dart flies on in a straight line and hits every enemy it crosses; damage/rate multiply the base stat (rate 1.8 = 1.8x slower); targets 2 = fires at the 2 first enemies; cluster 3 = 3 extra half-damage blasts around the landing point; burn = a fire patch at the landing point dealing dps to ground enemies for time seconds; vulnerable = enemies slowed by this tower take +25% damage from everything; stun = each enemy entering the aura has the chance to stop for time seconds (once per enemy per tower); shieldBreak 3 = Arc does 3x damage to shields; chains 6 replaces chains.
- Selling frees the pad (and the build price does not go back down).
Do not change unrelated code; list what you changed.
```

Test: upgrade a Bolt twice (damage then range circle grows), pick Twin: it shoots two enemies; Railgun is now locked. Sell it: the refund is 70% of the total spent and the pad is empty again.

P11. Overcharge

```
Add OVERCHARGE to settings.js: { rateMult: 3, duration: 5, cooldown: 30, key: 'Q' }.
- A round ability button at the bottom-right (purple, lightning icon, label "Q"). Pressing Q or clicking it arms Overcharge: the cursor shows a purple ring and the hint "Click a tower". Clicking a tower then gives it rateMult x fire rate (Frost: rateMult x aura damage) for duration seconds of game time; clicking anything else or pressing Q again cancels.
- While overcharged the tower glows (pulsing purple ring) and its shots are a bit bigger. Play a short rising beep (Web Audio, no files).
- Then the button shows a cooldown sweep (a dark pie that shrinks) for cooldown seconds of game time and ignores input.
Do not change unrelated code; list what you changed.
```

Test: press Q, click a Bolt: it fires about 3x as fast for 5 s; the button sweeps for 30 s; Q does nothing until it is ready.

P12. Juice and sound

```
Add juice, keeping the road and range circles readable (effects under 300 ms, never covering the road for long):
- Enemies flash white for 60 ms when hit (setTintFill white, then clear).
- Small white damage numbers float up 12 px and fade (only for hits of 5+ damage, max 30 on screen).
- Gold "+N" pops at each kill, drifting toward the gold counter.
- Mortar landings: a gold ring burst and a light camera shake (60 ms, 0.003). No other tower shakes the camera.
- Core stolen: red flash on the vault; core lost: a red camera flash 150 ms; core returned: green sparkle.
- src/sfx.js with tiny Web Audio blips: bolt shot, mortar boom, arc zap, kill, core stolen, core returned, wave start, build/sell. Limit each sound to 1 per 50 ms. M toggles mute (saved in localStorage) in GameScene only.
Do not change unrelated code; list what you changed.
```

Test: a busy wave 10 still runs at about 60 fps; every event has a sound; M mutes and survives a refresh.

P13. HUD polish, speed, pause, win and lose

```
- HUD top-left: cores, gold, wave (as before). Top-right: a 1x/2x speed button (sets this.speed, which already scales tick) and a pause button (P also toggles; dims the screen and shows PAUSED). Esc first closes whatever is open (build menu, tower panel, armed Overcharge) and only toggles pause when nothing is open. Pausing must stop enemies, cooldowns, countdown and Overcharge, because they all run on tick.
- Surviving wave 20 (all enemies gone) goes to GameOverScene with won: true. Losing all cores goes there with won: false. Pass { won, wave, cores } explicitly. GameOverScene shows "YOU WIN!" or "GAME OVER", "Wave N", "Cores saved: N" and the best wave from localStorage (key 'core-keepers-best-wave', saved when higher).
- Restart from GameOverScene and the menu must start a fresh run: this.scene.start('GameScene', { fresh: true }) and reset every field in create() (gold, cores, wave, towers, speed 1, paused false, Overcharge ready).
- Menu shows "Best wave: N".
- Remove the 1-6 debug spawn keys.
Do not change unrelated code; list what you changed.
```

Test: 2x makes everything twice as fast, including fire rate; pause freezes everything; win, lose, and restart all work and the best wave is kept after a refresh.

Optional P14. Endless mode with upgrade cards

```
After winning, offer "Endless" on the win screen. Endless continues from wave 21 with the same buildWave rules (count stays at the cap, hp keeps growing). After every 5th endless wave, pause and offer 3 random cards from a CARDS list in settings.js (e.g. "+15% Bolt damage", "Frost slows 10% more", "+1 core", "Overcharge cooldown -5 s", "+60 gold", "Mortar splash +20%"), with 1 free reroll per offer. Taking a card applies it for the rest of the run. Endless runs save their best wave separately (key 'core-keepers-best-endless').
Do not change unrelated code; list what you changed.
```

Test: win, choose Endless, clear wave 25: 3 cards appear; Reroll works once; the chosen card's effect shows up.

Optional P15. Balance pass

```
Write scripts/balance.mjs (plain Node, imports src/settings.js) that prints a table: for each tower and each upgrade path, damage per second, DPS per gold spent, and range; and for waves 1, 5, 10, 15, 20 the total enemy hp (shields included) and the total kill gold. Do not change game code. Then tell me which tower is the best and worst value and suggest 3 number changes in settings.js (only suggest; do not apply them). Do not change unrelated code; list what you changed.
```

Test: `node scripts/balance.mjs` prints the tables; nothing in src/ changed.

Ship check: Play one full run at 1x. With the numbers above, our three automated bots (5 seeds each, 1x) gave: a *weak* player (only Bolts, no upgrades) loses all cores at wave 10; an *average* player (all four towers, tiers 1-2, no forks or Overcharge) loses at wave 18-19; a *good* player (counters, forks, Overcharge) wins with 3-5 of 8 cores left. If your changes make the game much easier or harder, retune WAVES.hpGrowth first, then ECONOMY.startGold.

Universal prompts (any genre)

Add a pause menu

```
Add a pause menu toggled with Esc or P: dim the screen, show Resume / Restart / Menu buttons. Physics and timers must stop while paused.
```

Tune the feel

```
List every gameplay number in config.js with one line explaining what it does, and suggest which 3 to tweak to make the game feel <faster / heavier / easier>.
```

Clean up before shipping

```
Remove all console.log and debug keys. Check for objects that are created but never destroyed. Do not change gameplay. Then confirm npm run build passes.
```

Swap placeholder art

```
I put these files in public/: <hero.png 32x32, coin.png 16x16>. Load them in a preload() method and use them instead of the generated textures with the same keys. Keep the generated textures as fallback for anything without a file.
```

Grow Your Game: Add Content Without Breaking It

Browser Game Starter Kit by Prompt Forge Kits

You shipped v1. People played it and said "more!" This chapter shows how to add levels, waves, enemies, weapons, items, and new modes with an AI coding tool, without the game turning into a pile of patches.

It is written against the real starter in `starter/`. Where a snippet is shown, we built it into the starter and played it in a browser to check it (noted below). The numbers in examples are **placeholders to tune**, not recommendations. The only balance numbers we quote as tested are the Core Keepers ones already in `PROMPTS.md`.

1. What the starter gives you (and what it doesn't)

Read this once so your prompts match the code:

Where	What lives there
`src/config.js`	Every tunable number, `PALETTE`, `COLORS`, `TEXT`, `PLAYER`, `ENEMY` (patrol distance and duration), `SCORE`, `STORAGE_KEYS`, and the `INFO` object (every on-screen name and one-line description)
`src/textures.js`	All art: one draw function per sprite, registered with `save('key', w, h, ...)` inside `createTextures()`
`src/scenes/BootScene.js`	Builds textures and the animations (`anim(key, frameKeys, fps)`), loads the font
`src/scenes/GameScene.js`	**One hard-coded level**: the arrays `PLATFORMS`, `COINS`, `ENEMIES`, `SPIKES`, `GOAL` at the top, in tiles (1 tile = `TILE` = 36 px)
`src/ui.js`	`addText`, `addPanel`, `addTag`, `addLegend`, `addBanner`, `addTooltip`, parallax helpers

Two facts shape everything below:

1. **Level content is already data** (the arrays), but only one level exists and it lives inside the scene. Step one of growing is moving it into its own file.
2. **Every new thing needs three entries:** a draw function (`textures.js`), a behavior (a scene or `src/` file), and an on-screen name plus one-line description (`INFO` in `config.js`). `AGENTS.md` already tells your AI this; remind it when it forgets.

2. The growth rules

1. **One prompt = one content addition.** One enemy, one level, one item. Not "add three enemies and a boss".
2. **Tables first, content second.** Build the table (section 3) in one prompt. After that, adding content is adding a row.
3. **Play the new thing alone before combining.** Make a throwaway level or room with only the new enemy in it.
4. **Commit before and after every addition.** `git add -A && git commit -m "add runner enemy"`.
5. **Cap the scope.** Write in `GDD.md` section 7 what v2 will NOT have. Content is endless; your time is not.

6. **If a prompt fails three times, roll back** (`FIX-LOOP.md`) and split it into two smaller prompts.

The prompt shape that works for content

```
Add <ONE thing>. Files you may change: <list>.
It must have: a draw function in textures.js in the Chunky Flat Toy style (THEME.md, palette colors only),
an entry in <the table>, and an INFO name + one-line description in config.js.
Do not change unrelated code. List what you changed and tell me how to test it.
```

Telling the AI which files it may touch is the single best way to stop a 40-line request becoming a 400-line rewrite.

3. Make content data, not code

If adding an enemy means editing five methods, the AI will eventually break one. If it means adding a line to a table, it can't. The kit already uses this pattern: Tower Defense keeps every number in `src/settings.js` , Idle keeps upgrades in `src/economy.js` , Puzzle keeps levels as strings in `src/levels.js` , and Platformer prompt P7 moves levels to `src/levels.js` .

Levels as data (checked in the starter)

We moved the arrays out of `GameScene.js` into `src/levels.js` :

```
// src/levels.js: all level content is data. Add a level = add an object.
// Platforms: [tileX, tileY, widthInTiles]. Everything else: [tileX, groundRow].
export const LEVELS = [
  {
    name: 'Level 1',
    platforms: [[0, 11, 14], [17, 11, 12], /* ... */],
    coins: [[8, 8], [9, 8], /* ... */],
    enemies: [[9, 11], [21, 11] /* ... */],
    spikes: [[27, 11] /* ... */],
    goal: [65, 11]
  },
  { name: 'Level 2', /* same fields, new numbers */ }
];
```

`GameScene` then reads `this.level = LEVELS[this.levelIndex]` instead of the old constants, takes `levelIndex` and `score` in `init(data)` , and on the flag either starts `GameScene` with `{ levelIndex: next, score: this.score }` or calls `endGame(true)` after the last level. `MenuScene` and `GameOverScene` must start a fresh run with explicit data (`{ levelIndex: 0, score: 0 }`), because `scene.start()` without data can reuse the old data (see the pitfalls in `AGENTS.md`).

We built this with a second level and played it headless: menu, level 1, flag, level 2, flag, the end screen, restart back on level 1, with no console errors. The prompt that does it is in section 12 (Platformer). Note that `PROMPTS.md` P7 already contains it, so if you followed the playbook you have this.

Enemy kinds as a table (checked in the starter)

Right now `ENEMY` in `config.js` holds one patrol setting for all walkers. Turn it into one entry per kind:

```
// config.js: one entry per enemy kind
export const ENEMY_TYPES = {
  walker: { patrolDistance: 2 * TILE, patrolDuration: 1400 },
  runner: { patrolDistance: 3 * TILE, patrolDuration: 800 } // shorter duration = faster
};
```

and let each level row name its kind: `enemies: [[9, 11], [15, 11, 'runner']]` (the third value defaults to `'walker'`). We did exactly this (the runner row has a shorter duration, so it moves faster) and the game built and ran through all levels with no console errors. A real new enemy should also get its own look (section 5), not a recolor.

The rule for every table

- Plain numbers and short strings only (no functions). Easy for the AI to extend and for you to read.
- One name per row, used everywhere (`'runner'`, not three different spellings).
- Display text lives in the table row or in `INFO`, never inline in scenes.
- Ask the AI to **validate on load** in dev: `console.warn` if a level row names an enemy kind that doesn't exist. A one-line check saves a black screen.

4. Levels and waves

Platformer levels. Add one object to `LEVELS` per prompt. Ask the AI to keep every platform gap jumpable: with the starter's `PLAYER.speed` (220) and `PLAYER.jumpVelocity` (-460) and `WORLD.gravity` (900), measure the real jump by testing in game, then write the max gap (in tiles) at the top of `levels.js` as a comment and tell the AI never to exceed it.

Roguelite rooms and runner chunks. Don't hand-place everything. Give the AI a table of room templates or obstacle "chunks" and let a function pick from them, so more content = more templates.

Tower defense waves. The kit's `WAVES` table in `src/settings.js` already drives `buildWave(n)` (count, hp growth, unlock waves, boss waves). To grow: extend `WAVES.total`, add an enemy to `ENEMIES`, and add its wave to `WAVES.unlock`. No wave code changes.

Puzzle levels. One string array per level in `levels.js` (`'#'` wall, `'.'` floor, `'P'` player, `'B'` box, `'T'` target). The cheapest content to add, and the easiest to get wrong: ask the AI to add a solver check or have you test-solve every level before committing.

Pacing a set of levels:

- Introduce one new thing per level, in a safe spot, before using it dangerously. (The kit's wave builder does this: an enemy's first appearance is alone, with a one-line tip.)
- Alternate hard and easy. Two hard levels in a row is where players quit.
- Put the new level's mechanics on screen using the `INFO` helpers (`addBanner` for the goal, `addTag` for the first of each new thing).

5. New enemies

A new enemy is four small edits. Do them as **one** prompt per enemy:

1. **Draw function** in `textures.js` + a `save('enemy-runner-0', w, h, ...)` line in `createTextures()`. Two frames if it animates, and an `anim(...)` line in `BootScene.createAnimations()`.
2. **Table row** (`ENEMY_TYPES` or `ENEMIES`).

3. **Behavior** if it isn't a patroller: a small method or a new file in `src/` (for example `src/systems/`), not a rewrite of `GameScene`.
4. `INFO` **entry**: name and one line ("Runner: fast, avoid").

Make it read at a glance (THEME.md): a new enemy gets a distinct **shape and speed or size**, not only a new color. All enemies stay `red` (red = danger); the kit's shape language is circle (basic), triangle (fast), square (tanky), diamond (flying), hexagon (boss). Don't tint a baddie gold: gold means money.

Behavior ideas that need no new engine: patroller (starter), faster patroller, jumper (tween on `y`), shooter (fires every N ms, bullets destroyed off-screen), chaser (moves toward the player), armored (takes N hits), splitter (spawns two small ones on death). Ask for one per prompt.

6. Weapons and towers

- **Every weapon is a table row of numbers**: damage, rate (seconds between shots), range, speed, plus a text role line. The Core Keepers kit does this in `TOWERS` and in the combat stats of `settings.js`.
- **Give each one a job**, not just bigger numbers. Core Keepers has Bolt (fast single target, hits flyers), Mortar (splash, ground only), Frost (slows), and Arc (chain, ignores armor). Counter-relationships ("brutes shrug off Bolt, Arc handles them") make a roster worth having.
- **A new weapon must be worse at something.** Ask the AI: "What is this weapon bad at?" If the answer is "nothing", cut a stat.
- **Color = damage type**, and its projectiles use the same color (THEME.md): cyan bolt, blue frost, purple arc, gold splash.
- **Upgrades beat more towers.** Tiers plus a fork choice (the kit's `UPGRADES` table) add depth with no new art.
- **Every number in** `settings.js`, nothing hard-coded in the tower code. Otherwise balancing means hunting through files.

7. Items, pickups, and upgrades

- **Pickups** (platformer, runner): one new sprite plus one row. Reuse the starter's pattern: a physics group, an overlap callback, and a burst of `PARTICLES` in the pickup color. Guard one-time effects (overlaps fire every frame; see `isOver` in `GameScene`).
- **Upgrade cards** (roguelite, endless modes): keep them as a list of `{ name, text, apply }`-style entries, or plain data the game reads. Show three at random, one pick. The kit's Roguelite P7 and Tower Defense endless cards both work this way.
- **Colors by rarity:** common cards on `slateDark` panels, rare cards with a purple border (THEME.md: purple = magic/rare).
- **Every item shows its name and effect** ("Rapid Fire: +20% fire rate"). That is a rule in `AGENTS.md` (`INFO`); don't let the AI skip it.
- **Persistence:** if a run's upgrades should survive between rooms, put them in one object and pass it through the scene data. Anything saved across sessions uses a key from `STORAGE_KEYS`.
- **Avoid stacking explosions.** Percent bonuses that multiply each other get silly fast. Ask the AI to cap each stat in `config.js` (`MAX_FIRE_RATE`, and so on) and say so out loud in the prompt.

8. New modes

A mode is a scene plus a rule change, not a new game. Examples, in order of effort:

Mode	How to add it	Effort
Endless / survival	Reuse the content tables, keep generating after the last level or wave, track best in `STORAGE_KEYS`	Low
Time attack	Add a timer to the HUD (`addText` , labeled "Time: ..."), best time in localStorage	Low
Daily challenge	Seed your random picks with the date, so everyone gets the same run that day	Medium
Hard mode	A multiplier object in `config.js` (enemy speed, hp, fewer pickups) read by the same scenes	Low
Level select	A new `LevelSelectScene.js` : panels from `addPanel` , saved "unlocked up to" in localStorage	Medium
Local 2-player	Only if you planned it in the GDD; it touches input, camera, and win rules	High

Rules for adding a scene: one scene per file, class name = file name = scene key, register it in `src/main.js` , start it with explicit data, and add a way back to the menu. Test the full loop menu -> mode -> game over -> menu every time.

9. Keep the "Chunky Flat Toy" look as you grow

The fastest way for a game to look cheap is mixed art. Bake these into your prompts (or just say "follow THEME.md"):

- **Colors only from `PALETTE`** (16 names, no new hex values). New thing, new job? Use the role color: red danger, gold money/reward, cyan/blue player/friendly/slow, green heal/poison, purple magic/rare.
- **3 px `ink` outline** (`OUTLINE`) on every actor, pickup, and panel; one white highlight at 40-60% alpha; big eyes on characters.
- **Text via `addText()`** , panels via `addPanel()` . Don't let the AI create raw `this.add.text` .
- **New backgrounds stay muted**; actors stay bright. A new world theme = a new pair of `bg-far` / `bg-near` draw functions with muted palette colors.
- **Status effects are colored rings**: blue slow, green poison, purple magic.
- **One more review check:** screenshot the game. If a new thing doesn't pop against the background, make the background quieter, not the thing louder.

10. Balance, without math degrees

Most of this is playtesting. These are rules of thumb, not measured results:

- **Change one number at a time.** Then play. If you change three, you'll never know which one fixed it.
- **Keep all numbers in one place** (`config.js` , `settings.js` , `economy.js`). Balance sessions are then "edit values, reload".
- **Find the lever that scales everything.** In Core Keepers it is `WAVES.hpGrowth` , then `ECONOMY.startGold` . Every genre has one: enemy speed ramp (runner), cost growth (idle), enemies per room (roguelite). Tune that first.
- **Make a throwaway cheat to reach late content fast**: a debug key that jumps to level N or wave N. Ask the AI for it, and remove it before shipping.
- **Compare against a baseline.** Before adding content, note how far a typical run goes (rooms, waves, distance). After, check it hasn't jumped or collapsed.
- **Ask the AI for a balance table, not a verdict.** The kit's optional Tower Defense P15 writes `scripts/balance.mjs` , which prints DPS, DPS per gold, and wave totals. You can ask for the same for any genre: "write a script that prints each item's numbers side by side. Don't change game code."

- **Watch for dominant strategies.** If every winning run uses the same weapon, nerf that one or give the others a niche.
- **Reference points from the kit.** Core Keepers' ship check (in `PROMPTS.md`) records what its automated bot runs produced with the shipped numbers (weak player loses at wave 10, average at 18-19, good player wins). If your additions move those results a lot, retune `WAVES.hpGrowth` first. Your own game needs its own reference runs; write yours down.

11. The playtest loop for every addition

1. **Isolate.** Create a tiny test level, room, or wave with just the new content.
2. **Play it for two minutes.** Does it read? Is it clear what it is and what to do? Is it fair?
3. **Check the console (F12).** Red errors mean stop and fix.
4. **Play the normal game once** to make sure nothing else changed.
5. **Watch someone else.** Silence and don't explain. Where they die or get confused is your next prompt.
6. **Commit** with a message naming the addition. Remove the test level only after committing.

A short list to keep in your notes: *Is the new thing named on screen? Does it use the theme colors? Does restart reset it? Do old objects get destroyed? Does* `npm run build` *still pass?*

12. Example prompts by genre

Paste Prompt 0 from `PROMPTS.md` first so the AI knows your project. Then use these. Text in `<angle brackets>` is yours. Do them one at a time, testing between.

Platformer (starter as-is)

Add a level

```
Add one new level to src/levels.js (create it from the arrays at the top of GameScene.js first if it doesn't exist; keep level 1 identical). Level 2 uses the same fields as level 1, has at least one gap I can still clear with the current jump, and ends at the flag. Do not add new enemy kinds. Only change src/levels.js, GameScene.js, MenuScene.js and GameOverScene.js, and start every new run with explicit data { levelIndex: 0, score: 0 }. Tell me how to test the full path menu -> level 1 -> level 2 -> win.
```

Add an enemy kind

```
Add a "jumper" enemy. Draw function in textures.js (a red triangle with big eyes, outline and highlight per THEME.md, 2 frames), registered with save() and an animation in BootScene. Add it to an ENEMY_TYPES table in config.js (patrol distance, duration) and let a level row choose its kind. Add an INFO name: "Jumper: stomp or dodge". Use it in level 2 only, once. Don't change other enemies.
```

Add a pickup

```
Add a purple "Spring Shoes" pickup that doubles jump height for 8 seconds. One draw function (purple, THEME.md), one entry in config.js for the duration and multiplier, INFO name "Spring Shoes: high jump (8 s)", a tag on the first one on screen, a purple burst when collected, and a HUD label while it's active. Reset the effect in create().
```

Top-down roguelite

Add an enemy

```
Add a "dasher" enemy: a red triangle that pauses for 0.6 s, then dashes toward where the player was, then recovers. Put its hp, speed, pause, and dash numbers in an ENEMY_TYPES table in config.js. Wave size and position logic stay as they are; the room spawner picks a type from a list of {type, minRoom} so the dasher appears from room 3. INFO name: "Dasher: dodge the rush". Only change textures.js, config.js, the enemy file, and the room spawner.
```

Add upgrade cards

```
Add two new cards to the upgrade pool (data only, no new systems): "Piercing Shots" (bullets pass through one enemy) and "Quick Feet" (+10% move speed, max +30% total). Each card shows its name and one-line effect, rare cards have a purple border. Put the cap in config.js. Don't change how cards are offered.
```

Add a room type

```
Add a "treasure room" type: no enemies, three gold coins worth <N> score each, and a door that is already open. Every 4th room is a treasure room after room 3. Label it with a banner: "Treasure room: grab the coins". Keep the boss-room rule from earlier unchanged.
```

Idle / clicker

Add a generator

```
Add one more generator to the data table in src/economy.js: "<Name>", <coins/s> per second, base cost <cost>, same 1.15x cost scaling as the others. Do not change any other row or any formula. The shop shows it automatically with its one-line description built from its numbers, and the tooltip works. Use only data, no new code paths.
```

Add prestige

```
Add a prestige button that appears once total coins earned reaches <threshold>. Pressing it resets coins and generators, and gives permanent "stars" (purple, magic/rare) that add +<N>% to all income each. Save stars in localStorage with a key in STORAGE_KEYS. Show stars in the HUD with a label. Put every number in economy.js.
```

Add an achievement

```
Add 3 achievements as a data list in economy.js ({ name, text, check }) shown in a small panel: "First Helper", "100 clicks", "1,000 coins". When one unlocks, show a banner and save it in localStorage. Don't change income math.
```

Endless runner

Add an obstacle

```
Add a "flyer" obstacle: a red diamond that moves left at the world speed at jump-over-or-duck height. Draw it in textures.js (THEME.md: diamond = flying), add it to an obstacle table in config.js (create one if there isn't one yet) with its spawn weight, and make it unlock after <N> meters. INFO tag on the first one: "Flyer: duck!". Keep the spacing rule that guarantees a clear jump between obstacles.
```

Add a pickup

```
Add a purple "Shield" pickup that absorbs one hit (the obstacle disappears instead of ending the run). It appears at most once every <N> meters. Show a blue ring around the player while active and a HUD label "Shield: ready". Put the spawn gap in config.js.
```

Add a biome

```
Add a second background theme after <N> meters: new bg-far and bg-near draw functions in textures.js using muted palette colors only, cross-faded over 2 seconds. Obstacles and the player must stay clearly brighter than the background.
```

Puzzle (Sokoban-style)

Add levels

```
Add 3 levels to src/levels.js using the same characters as the existing levels ('#', '.', 'P', 'B', 'T'). Each must be solvable, smaller than 10x8, and teach one idea: <idea 1>, <idea 2>, <idea 3>. Also add a dev-only check that logs an error if a level has a different number of boxes and targets, or no player. Don't touch rendering.
```

Add a tile type

```
Add a "one-way floor" tile (character '>'): the player and boxes can enter it only moving right. Draw it in textures.js (slate with a gold arrow, THEME.md), parse it in levels.js, add it to INFO, and make undo still work with it. Add one level that uses it.
```

Tower defense (Core Keepers)

Add an enemy

```
Add one enemy "<name>" to ENEMIES in settings.js: hp <n>, speed <n>, size <n>, shape '<shape not yet used for this role>', gold <n>, and its name + one-line text (8 words or fewer). No new code unless the behavior needs it; if it does, list the exact change first. Add its unlock wave to WAVES.unlock and a one-line tip on its first appearance. Run scripts/balance.mjs before and after and show me the difference.
```

Add a tower

```
Add one tower "<name>" to TOWERS in settings.js (cost, color by damage type from THEME.md, shape, role, desc, damage, rate, range, hitsFlyers) and one case in drawTowerIcon(). It must have a clear weakness: <weakness>. Do not change other towers. Test: it appears in the radial menu with its info card, and fires correctly.
```

Add waves or a map

```
Raise WAVES.total from 20 to <N>, and add one boss wave to bossWaves. Do not change hpGrowth. Then write down (don't apply) three number changes in settings.js you'd try if the extra waves are too easy. Add a second map as a new file with its own PATH and PADS in the same format as map.js, and a way to pick it from the menu.
```

13. Content checklist (copy this)

For every new thing you add:

- [ ] One prompt, one addition, committed before and after
- [ ] A row in a table (levels, enemies, towers, items, cards), not new code in five places
- [ ] A draw function in the Chunky Flat Toy style, palette colors only
- [ ] A name and one-line description on screen (`INFO`)
- [ ] Played alone first, then in the normal game
- [ ] Console clean, restart works, `npm run build` passes
- [ ] Balance changed one number at a time, noted in your commit message

And finally: **content is easy to add and hard to finish.** Pick a number (three levels, two new enemies, one mode), hit it, publish with `PUBLISH.md`, and ask your players what they want next.

The Fix Loop: Recovering When AI Breaks Your Game

AI tools are fast at writing code and also fast at breaking it. This guide is the habit that keeps you shipping instead of spiraling.

The golden rule: 3 tries, then roll back

```
Change breaks something
  -> Try 1: paste the exact error + what you did (Template A)
  -> Try 2: give more context: file contents, what you expected (Template B)
  -> Try 3: ask the AI to explain the cause BEFORE fixing (Template C)
  -> Still broken? ROLL BACK to your last commit. Re-ask with a smaller step.
```

Why: after 3 failed fixes the AI is usually patching its own patches. Each attempt adds code, and the bug gets buried deeper. Rolling back costs you 2 minutes; spiraling costs you an evening.

Git checkpoints (your save points)

Run once at the start of a project:

```
git init
git add -A
git commit -m "starter works"
```

After every working step (a feature works and you tested it):

```
git add -A
git commit -m "player can double jump"
```

See your save points:

```
git log --oneline
```

Undo all uncommitted changes (go back to the last save point):

```
git checkout .          # restore changed files
git clean -fd           # remove new untracked files (careful: deletes them)
```

Or with newer git: `git restore .`

See what the AI changed since the last commit:

```
git diff
git status
```

Go back to an older save point (keeps history, safe):

```
git log --oneline            # find the commit id, e.g. a1b2c3d
git revert --no-edit HEAD    # undo the most recent commit with a new commit
```

Or just look at an old version without changing anything:

```
git checkout a1b2c3d         # look around, run npm run dev
git checkout main            # come back (or "master" on older setups)
```

Try something risky on a branch:

```
git checkout -b experiment   # work here
git checkout main            # abandon it: just switch back
git merge experiment         # keep it: merge into main
```

Tip: Cursor has a "Restore checkpoint" button and Claude Code has `/rewind` (Esc twice); Lovable and Bolt have version history. Use them too, but git is the one that always works.

Before you ask the AI: 60-second triage

1. **Open the browser console** (F12 or Cmd+Option+J, "Console" tab). Red text is your error. Copy all of it.
2. **Check the terminal** running `npm run dev` for build errors.
3. **Hard refresh** (Ctrl+Shift+R / Cmd+Shift+R). Sometimes it's just stale.
4. **Name the symptom precisely:** "black screen", "player falls through floor", "score doesn't reset on restart". Vague in = vague out.

Debug prompt templates

Template A: error message (Try 1)

```
After your last change the game is broken.
What I did: <e.g. pressed Space on the menu>
What I expected: <the game starts>
What happened: <black screen>
Console error (exact):
<paste the full red error text>
Fix only this bug. Change as little code as possible and tell me which files you changed and how to
test the fix.
```

Template B: no error, wrong behavior (Try 2)

```
There is no error, but the behavior is wrong.
Expected: <coins disappear when touched and score goes up by 10>
Actual: <score goes up by hundreds per coin>
Relevant file: src/scenes/GameScene.js
Here is the current code of the function involved:
<paste the function>
Explain in one paragraph why this happens, then make the smallest fix.
```

Template C: diagnose before fixing (Try 3)

```
We have tried to fix this twice and it is still broken. Do NOT write code yet.
1. List the 3 most likely causes of: <symptom>.
2. For each, tell me one thing I can check (a console.log to add or a value to look at).
Wait for my results before changing code.
```

Template D: add logging to find the bug

```
Add temporary console.log statements to trace <the problem>, e.g. log the player's position and velocity every 30 frames, and log when <event> fires. Prefix each log with [DEBUG] so I can remove them later. Don't change any game logic.
```

Template E: after rolling back (fresh start)

```
I rolled back to my last working commit. Previously, trying to add <feature> in one step broke <thing>.
Let's do it in smaller steps. Give me just step 1 of <feature>: the smallest change that is visible and testable. Stop after that step.
```

Template F: "it was working before"

```
Here is the output of `git diff` since the last working version:
<paste diff>
Something in this diff broke <thing>. Identify which change caused it and revert only that part.
```

Template G: performance (game is slow/laggy)

```
The game gets slow after <30 seconds / many enemies>. Check for: objects created every frame and never destroyed, tweens or timers that pile up, listeners added in update(). Show me what's leaking and fix it with minimal changes.
```

Classic Phaser bugs and quick fixes

Symptom	Usual cause	Fix
Black screen, no error	Scene key typo / scene not in `main.js` list	Check `super('Key')` matches `scene.start('Key')`
Things happen hundreds of times	Overlap callback runs every frame	Guard with a flag or destroy/disable the object
State not reset after restart	Variables set in constructor	Reset them in `create()`
Player falls through platforms	Missing `physics.add.collider` / very high speed	Add the collider; lower speed or make platforms thicker
Image is a green box with a slash	Wrong asset path or key	Files in `public/` load as `'name.png'`; check key spelling
Works in dev, blank on itch.io	Absolute paths	Keep `base: './'` in `vite.config.js`, no leading `/` in asset paths
HUD scrolls away	Camera follows player	`.setScrollFactor(0)` on HUD objects
Input stops working after restart	Listeners added with `on` stack up	Use `once`, or remove listeners in `shutdown`

When the AI keeps "fixing" things you didn't ask for

```
Stop. Only change what I ask. Do not refactor, rename, or reformat other code. Re-read AGENTS.md and follow the "small steps" rules.
```

Publishing Your Game

Three ways to put your game online, all free. itch.io is best for players and feedback; GitHub Pages and Netlify are best for a simple link.

Step 0: Build and test (always)

```
npm run build      # creates dist/
npm run preview    # open the URL it prints and play a full run
```

Checklist before uploading:

- [ ] Menu -> game -> game over -> restart works
- [ ] No red errors in the browser console (F12)
- [ ] `vite.config.js` still has `base: './'` (needed for itch.io and GitHub Pages)
- [ ] Page title in `index.html` is your game's name
- [ ] You own or have a license for every image/sound

Option A: itch.io (recommended)

1. **Zip the build.** Zip the *contents* of `dist/`, so `index.html` sits at the top level of the zip, not inside a `dist` folder.
   - Mac: open `dist/`, select all files, right-click > Compress.
   - Windows: open `dist/`, select all, right-click > Send to > Compressed (zipped) folder.
   - Terminal: `cd dist && zip -r ../game.zip . && cd ..`
2. Create a free account at **itch.io**, then go to **Dashboard > Create new project**.
3. Fill in:
   - **Title**, **Project URL**, short **description**.
   - **Kind of project: HTML** (important).
4. **Uploads:** upload `game.zip` and tick **"This file will be played in the browser"**.
5. **Embed options:**
   - Viewport: 800 x 450 (the starter's size) or larger; tick **"Mobile friendly"** if you support touch, and **"Fullscreen button"**.
   - Tick **"Automatically start on page load"** only if your game has a menu (the starter does).
6. Add a **cover image** (630 x 500) and 3+ **screenshots**. Pages with screenshots get far more clicks.
7. **Genre + tags** (e.g. platformer, pixel-art, short, made-with-phaser).
8. **Visibility:** start with **Draft**, click "View page", play it. Then set to **Public** and **Save**.

Updating: build again, re-zip, upload the new zip and delete the old one (or use itch's `butler` CLI later).

Troubleshooting: blank game on itch = `index.html` is not at the zip root, or asset paths start with `/`.

Option B: GitHub Pages

1. Create a GitHub account and a new **public** repository (e.g. `my-game`).
2. Push your project:

```
git remote add origin https://github.com/YOURNAME/my-game.git
git branch -M main
git push -u origin main
```

3. Add the automatic deploy workflow. Create `.github/workflows/deploy.yml`:

```
name: Deploy to GitHub Pages
on:
  push:
    branches: [main]
permissions:
  contents: read
  pages: write
  id-token: write
jobs:
  deploy:
    runs-on: ubuntu-latest
    environment:
      name: github-pages
      url: ${{ steps.deployment.outputs.page_url }}
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
      - run: npm ci
      - run: npm run build
      - uses: actions/upload-pages-artifact@v3
        with:
          path: dist
      - id: deployment
        uses: actions/deploy-pages@v4
```

4. On GitHub: **Settings > Pages > Build and deployment > Source: GitHub Actions**.
5. Commit and push the workflow:

```
git add -A && git commit -m "deploy to pages" && git push
```

6. Watch the **Actions** tab. When it's green, your game is at `https://YOURNAME.github.io/my-game/`.

Every future `git push` redeploys automatically.

Option C: Netlify

Fastest (drag and drop):

1. Run `npm run build`.
2. Go to **app.netlify.com/drop** (log in or sign up).
3. Drag the **`dist` folder** onto the page. You get a live URL in seconds.
4. Rename the site under **Site configuration > Change site name**.

Auto-deploy from GitHub:

1. Push your project to GitHub (Option B, step 2).
2. Netlify: **Add new site > Import an existing project > GitHub**, pick the repo.
3. Build command: `npm run build`. Publish directory: `dist`. Deploy.
4. Every `git push` now redeploys.

After you publish

- Post your link with a GIF of gameplay (ScreenToGif / Kap / OBS) on X, Reddit (r/WebGames, r/IndieDev, r/playmygame), and itch.io's community.
- Join a game jam on itch.io: deadlines make you ship.
- Ask 3 people to play while you watch silently. Fix what confuses them first.

License

Browser Game Starter Kit v1.0 by Prompt Forge Kits: license for buyers.

Licensor: Prompt Forge Kits (promptforgekits.com). "You" means the person who bought the kit.

You MAY

- Use, modify, and build on the starter code (everything in `starter/`) in any number of your own games, **including commercial games** you sell, publish, or monetize (itch.io, Steam, web portals, ads, etc.).
- Publish the source code of your **finished game**, even if it contains parts of the starter code, as long as it is a game and not a kit, template, or tutorial.
- Use the prompts, templates, and guides to make your own games.

You may NOT

- Resell, share, or redistribute the kit itself, or any part of it as a kit: the starter project as a template, the prompt playbook, the guides, or `GUIDE.pdf`. This includes uploading it for free.
- Repackage it, in whole or in part, into another starter kit, template, course, or prompt pack.

Refunds

If the kit didn't work for you, you can request a refund within 7 days of purchase. Refunds may be declined if the kit has clearly been copied or reused. Updates are free for life.

No warranty

The kit is provided "as is", without warranty of any kind. Third-party tools and services it mentions (Phaser, Vite, itch.io, GitHub, Netlify, AI tools) are covered by their own licenses and terms. Phaser and Vite are MIT-licensed open-source projects and are not part of this license. The Lilita One font in `starter/public/assets/fonts/` is under the SIL Open Font License 1.1, which lets you bundle it with your games, including commercial ones (see `starter/CREDITS.md`). The starter's art is drawn in code and is part of the starter code.

Questions about a use not covered here? Contact Prompt Forge Kits at promptforgekits.com, email **[TODO: support email]**, or the Gumroad product page.

© 2026 Prompt Forge Kits. All rights reserved, except as granted above.